



Departamento de Informática e Matemática Aplicada – DIMAp

Conceitos Avançados do GraphQL

Prof. Nélio Cacho

Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.
- Não está autorizada a gravação ou reprodução desta aula

Operações de Mutação no GraphQL

- Mutações são operações para modificar dados no servidor.
- Pense nelas como equivalentes a POST, PUT, PATCH e DELETE em uma API REST.
- Ao contrário das queries, que apenas buscam dados, as mutações alteram o estado do backend.
- Mutações são declaradas com a palavra-chave **Mutation**.
- Elas podem receber argumentos de entrada (idealmente usando tipos input) e, como queries, retornam dados após a operação.

```
input AlunoInput{
  idAluno: ID!
  primeiroNome: String
  sobrenome: String
  idGrupo: Int
}

type Mutation{
  criarAluno(aluno: AlunoInput!): Aluno
  atualizarAluno(id: ID!, aluno: AlunoInput!): Aluno
  removerAluno(id: ID!): Aluno
}
```

Mudança no Controller

- A anotação `@MutationMapping` é usada para mapear métodos em controladores Spring diretamente para campos de mutação definidos em seu schema GraphQL.
- Quando o ***Spring for GraphQL*** recebe uma requisição GraphQL que contém uma operação de mutação, ele procura por métodos anotados com `@MutationMapping` cujo nome corresponda ao nome do campo de mutação no schema.

Mudança no Controller

```
@MutationMapping
public Mono<AlunoDto> criarAluno(@Argument AlunoDto aluno){
    return this.alunoService.criarAluno(aluno);
}

@MutationMapping
public Mono<AlunoDto> atualizarAluno(@Argument Integer id, @Argument AlunoDto aluno){
    return this.alunoService.atualizarAluno(id, aluno);
}

@MutationMapping
public Mono<AlunoDto> removerAluno(@Argument Integer id){
    return this.alunoService.deletarAluno(id);
}
```

Exemplo

```
1 query MyQuery {  
2   todosalunos {  
3     idAluno  
4     primeiroNome  
5     sobrenome  
6   }  
7 }
```



Variables Headers

1

```
{  
  "data": {  
    "todosalunos": [  
      {  
        "idAluno": "1",  
        "primeiroNome": "João",  
        "sobrenome": "Silva"  
      },  
      {  
        "idAluno": "2",  
        "primeiroNome": "Maria",  
        "sobrenome": "Souza"  
      },  
      {  
        "idAluno": "3",  
        "primeiroNome": "Thiago",  
        "sobrenome": "Luiz"  
      },  
      {  
        "idAluno": "4",  
        "primeiroNome": "Josefa",  
        "sobrenome": "Barbosa"  
      },  
      {  
        "idAluno": "5",  
        "primeiroNome": "Luiz",  
        "sobrenome": "Gonzaga"  
      }  
    ]  
  }  
}
```

Exemplo

```
1 mutation MyMutation {
2   criarAluno(
3     aluno: {idAluno: "6", primeiroNome: "Nelio", idGrupo: 10, sobrenome: "Cacho"}
4   ) {
5     primeiroNome
6     sobrenome
7     idGrupo
8   }
9 }
```



```
{
  "data": {
    "criarAluno": {
      "primeiroNome": "Nelio",
      "sobrenome": "Cacho",
      "idGrupo": 10
    }
  }
}
```

```
1 mutation MyMutation {
2   atualizarAluno(
3     aluno: {idAluno: "5", primeiroNome: "Ricardo",
4     id: "5"
5   ) {
6     primeiroNome
7     sobrenome
8     idGrupo
9   }
10 }
```



```
{
  "data": {
    "atualizarAluno": {
      "primeiroNome":
      "Ricardo",
      "sobrenome":
      "Luiz",
      "idGrupo": 10
    }
  }
}
```

Múltiplas Requisições de Mutação

- Pode-se enviar múltiplas requisições de mutação em uma única requisição, a diferença é que a execução é sequencial. Não é possível combinar operações de query com mutação.

```
1 mutation MyMutation {  
2  
3   criarAluno(  
4     aluno: {idAluno: "6", primeiroNome: "Nelio", idGrupo: 10  
5   } {  
6     primeiroNome  
7     sobrenome  
8     idGrupo  
9   }  
10  
11   atualizarAluno(  
12     aluno: {idAluno: "6", primeiroNome: "Ricardo", idGrupo:  
13       id: "6"  
14   } {  
15     primeiroNome  
16     sobrenome  
17     idGrupo  
18   }  
19 }
```



```
{  
  "data": {  
    "criarAluno": {  
      "primeiroNome": "Nelio",  
      "sobrenome": "Cacho",  
      "idGrupo": 10  
    },  
    "atualizarAluno": {  
      "primeiroNome":  
        "Ricardo",  
      "sobrenome": "Luiz",  
      "idGrupo": 10  
    }  
  }  
}
```


Subscrição em GraphQL

- É o **terceiro tipo de operação** GraphQL, ao lado de:
 - query: busca dados.
 - mutation: modifica dados.
 - subscription: escuta eventos em **tempo real**.
- Permitir que o cliente receba **atualizações automáticas** do servidor sempre que ocorrer uma mudança relevante.

Subscrição em GraphQL

- O campo `eventoAluno` é uma “fonte de dados contínua” — ou seja, o cliente se inscreve (`subscribe`) e o servidor **mantém uma conexão aberta** enviando atualizações sempre que há novos dados.

```
type Subscription{
  |   obtereventosAluno: EventoAluno
  | }

type EventoAluno{
  |   id: ID
  |   action: Action
  | }

enum Action{
  |   CREATED,
  |   UPDATED,
  |   DELETED
  | }
```

Subscrição em GraphQL

- **Fluxo conceitual:**
 - O cliente executa uma operação `subscription` via **WebSocket**.
 - O servidor abre uma **conexão persistente** (não fecha após a resposta).
 - Sempre que um novo evento ocorre (ex: novo aluno criado), o servidor **publica o evento**.
 - Todos os clientes inscritos recebem o novo dado **instantaneamente**.
- **O type `Subscription` é, portanto, um stream de dados contínuo — ele não termina enquanto a conexão estiver aberta.**

Subscrição em GraphQL

- No GraphQL, a **Subscription** representa um **canal contínuo de eventos futuros** — *não uma repetição do passado*. Portanto, o comportamento **"hot"** é o ideal — diferente de um *cold publisher* que reinicia o fluxo para cada cliente

```
@Service
```

```
public class EventosAlunoService {  
    private final Sinks.Many<EventoAluno> sink = Sinks.many().multicast().onBackpressureBuffer();  
    private final Flux<EventoAluno> flux = sink.asFlux().cache(history:0);  
  
    public void emitEvent(EventoAluno event){  
        this.sink.tryEmitNext(event);  
    }  
  
    public Flux<EventoAluno> subscribe(){  
        return this.flux;  
    }  
}
```

Subscrição em GraphQL

- A anotação `@SubscriptionMapping` é usada para implementar operações que retornam um fluxo contínuo de dados (Flux).
- O método retorna um `Flux<EventoAluno>`, ou seja, uma stream reativa de eventos. O Spring GraphQL usa essa stream para enviar atualizações em tempo real a todos os clientes conectados.

```
← EventosAlunoService
@Controller
public class EventoAlunoController {
    ← EventosAlunoService
    @Autowired
    private EventosAlunoService service;

    @SubscriptionMapping
    public Flux<EventoAluno> obtereventosAluno(){
        return this.service.subscribe();
    }
}
```

Subscrição em GraphQL

- Se o seu servidor estiver corretamente configurado com o endpoint de WebSocket (/graphql com graphql-ws habilitado), será possível ver os eventos chegando em tempo real conforme o método emitEvent() for chamado no backend.

```
spring.application.name=projetographql  
  
spring.graphql.graphiql.enabled=true  
spring.graphql.websocket.path=/graphql  
spring.graphql.path=/graphql
```

Subscrição em GraphQL

MySubscription2

+

```
1 subscription MySubscription2 {  
2   | obtereventosAluno {  
3   |   | action  
4   |   | }  
5 }  
}
```



GraphiQL



Subscrição em GraphQL

MyQuery MyMutation x +

```
1  mutation MyMutation {  
2  
3    criarAluno(  
4      aluno: {idAluno: "6", primeiroNome: "Nelio", ic  
5    } {  
6      primeiroNome  
7      sobrenome  
8      idGrupo  
9    }  
10  
11  
12 }
```



GraphQL

```
{  
  "data": {  
    "criarAluno": {  
      "primeiroNome":  
        "Nelio",  
      "sobrenome": "Cacho",  
      "idGrupo": 10  
    }  
  }  
}
```


Subscrição em GraphQL

MySubscription2

+

```
1 subscription MySubscription2 {  
2   | obtereventosAluno {  
3   |   | action  
4   |   | }  
5 }
```



GraphQL

```
{  
  "data": {  
    "obtereventosAluno": {  
      "action": "CREATED"  
    }  
  }  
}
```

Tratamento de Erros

- No modelo **REST tradicional**, o tratamento de erros segue o padrão **HTTP Status Code + corpo da resposta**.
- O **status HTTP** indica o tipo de erro (400, 404, 500, etc.).
- O **corpo da resposta** contém detalhes estruturados do erro (geralmente via `@ControllerAdvice + @ExceptionHandler`).
- A pipeline do **Spring WebFlux** (reativo) lida com `Mono.error()` ou `Flux.error()`, e o handler converte para `ResponseEntity` com status apropriado.

Tratamento de erros no GraphQL (Spring WebFlux)

- Em **GraphQL**, a resposta HTTP quase sempre é **200 OK**, mesmo quando há erros — porque o protocolo entende que a requisição GraphQL em si foi processada com sucesso (a query foi avaliada, mesmo que contenha falhas parciais).

Tratamento de erros no GraphQL (Spring WebFlux)

- O corpo JSON sempre possui **duas chaves possíveis**:
- data: resultado parcial ou nulo.
- errors: lista de erros.
- O status HTTP é **200 OK**, mesmo em erros de negócio.
- Somente erros **de infraestrutura** (como sintaxe GraphQL inválida, ou falha no servidor) retornam 4xx ou 5xx.

```
{
  "data": {
    "usuario": null
  },
  "errors": [
    {
      "message": "Usuário não encontrado",
      "path": ["usuario"],
      "locations": [{ "line": 2, "column": 3 }],
      "extensions": {
        "code": "NOT_FOUND",
        "timestamp": "2025-10-16T10:00:00Z"
      }
    }
  ]
}
```

Tratamento de erros no GraphQL (Spring WebFlux)

- O `@GraphQLExceptionHandler` é o **equivalente GraphQL do `@ExceptionHandler`** do Spring MVC ou WebFlux.
- Ele permite **interceptar exceções lançadas dentro da execução de uma query/mutation GraphQL e transformá-las em objetos `GraphQLError`** — que o Spring inclui automaticamente na resposta JSON, dentro do campo `errors`.
- Você usa esse annotation **dentro de um `@Controller GraphQL`** (ou seja, uma classe anotada com `@Controller` e com métodos como `@QueryMapping` ou `@MutationMapping`).

Tratamento de erros no GraphQL (Spring WebFlux)

```
@Controller
public class UsuarioController {

    private final UsuarioService service;

    public UsuarioController(UsuarioService service) {
        this.service = service;
    }

    // 📌 Método GraphQL normal
    @QueryMapping
    public Mono<Usuario> usuario(@Argument Long id) {
        // se o ID não existir, o service lança uma exceção
        return service.buscarPorId(id);
    }

    // 📌 Método que trata especificamente uma exceção
    @GraphQLExceptionHandler(UsuarioNaoEncontradoException.class)
    public GraphQLError handleUsuarioNaoEncontrado(UsuarioNaoEncontradoException ex) {
        return GraphQLErrorBuilder.newError()
            .message(ex.getMessage())
            .errorType(ErrorType.NOT_FOUND)
            .build();
    }
}
```

Unões em GraphQL

- Em GraphQL, cada *field* de uma query tem **um tipo de retorno fixo** — mas na prática, **nem sempre sabemos exatamente qual tipo virá**.
- Imagine uma busca global, ou uma timeline de rede social:
 - Um resultado pode ser um **usuário**,
 - um Comentário,
 - Um Story.

Unões em GraphQL

- **union** representa “**um campo que pode retornar múltiplos tipos diferentes**” — sem precisar de herança ou atributos compartilhados.
- Exemplo:
 - union ResultadoBusca = Usuario | Produto | Categoria
 - union ItemFeed = Post | Story | Video
 - union Pagamento = Pix | CartaoCredito | Boleto

Unões em GraphQL

```
union ResultadoBusca = Cachorro | Gato | Produto

type Query {
  animais: [Animal!]!
  buscaGlobal(texto: String!): [ResultadoBusca!]!
}
```

```
public Flux<Object> buscaGlobal(String texto) {
    return Flux.just(
        new Cachorro(3L, "Bolt", "Border Collie"),
        new Produto(10L, "Coleira Inteligente", 149.90),
        new Gato(4L, "Luna", "Azuis")
    );
}
```

```
@QueryMapping
public Flux<Object> buscaGlobal(@Argument String texto) {
    return service.buscaGlobal(texto);
}
```

Caching por requisição

- Esse é o **mecanismo ideal e recomendado** dentro do Spring GraphQL — e funciona muito bem com **Spring WebFlux**.
 - O **DataLoader**:
 - Agrupa múltiplas chamadas iguais dentro da mesma execução.
 - Mantém um cache **por requisição** (não global).
 - É **reativo**, e integra-se com o ExecutionInput do Spring GraphQL.
- O DataLoader é **resetado a cada execução de query GraphQL**. Isso evita:
 - Vazamento de memória (mantendo cache só enquanto a query é processada);
 - Erros de consistência (dados antigos sendo servidos em outra requisição);
 - Concorrência entre usuários.

Caching por requisição

```
@Component
public class PostDataLoader implements MappedBatchLoader<Long, List<Post>> {

    private final PostService postService;

    public PostDataLoader(PostService postService) {
        this.postService = postService;
    }

    @Override
    public Mono<Map<Long, List<Post>>> load(Set<Long> usuarioIds) {
        return postService.buscarPorUsuarioIds(new ArrayList<>(usuarioIds))
            .collectMultimap(Post::usuarioId)
            .map(HashMap::new);
    }
}
```

Caching por requisição

```
@Configuration
public class DataLoaderConfig implements DataLoaderRegistrar {

    private final PostDataLoader postDataLoader;

    public DataLoaderConfig(PostDataLoader postDataLoader) {
        this.postDataLoader = postDataLoader;
    }

    @Override
    public void registerDataLoaders(DataLoaderRegistry registry) {
        registry.register("postsLoader", DataLoaderFactory.newMappedDataLoader(postDataLoader));
    }
}
```

Caching por requisição

```
@Controller
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping
    public Flux<Usuario> usuarios() {
        return usuarioService.buscarTodos();
    }

    @SchemaMapping(typeName = "Usuario", field = "posts")
    public Mono<List<Post>> posts(Usuario usuario, DataLoader<Long, List<Post>> postsLoader) {
        // ✅ O DataLoader vai cachear chamadas repetidas na mesma requisição
        return postsLoader.load(usuario.id());
    }
}
```

O que acontece em tempo de execução

- O Spring GraphQL executa `usuarios()` e descobre que precisa dos posts de vários usuários.
- O DataLoader intercepta essas chamadas e **agrupa os IDs** em uma única chamada: Buscando posts para `[1, 2]`
- Os resultados são armazenados em um **cache in-memory por requisição**.
Qualquer novo acesso ao mesmo usuário reusa o resultado sem nova chamada.

```
{  
  usuarios {  
    id  
    nome  
    posts {  
      id  
      titulo  
    }  
  }  
}
```

Consumindo APIs GraphQL

- GraphQL é diferente de REST: O formato da query e da resposta exige um tratamento específico, não apenas um payload JSON simples.
- A biblioteca Spring for GraphQL unifica o desenvolvimento (servidor) e o consumo (cliente) de APIs GraphQL.
- O **GraphQLClient**: Define um fluxo de trabalho comum para requisições GraphQL, independente do transporte subjacente (HTTP, WebSocket, RSocket).

Consumindo APIs GraphQL

- O **HttpGraphQLClient** Implementação do GraphQLClient que utiliza o **Spring WebClient**.
- Aproveita a arquitetura **Reativa (Reactor)** e **Não Bloqueante** do WebClient.
- Ideal para microsserviços que precisam se comunicar com outros serviços GraphQL.

Consumindo APIs GraphQL

- A maneira mais comum de criar uma instância é a partir de um WebClient já configurado:

```
private final HttpGraphQLClient client;

public ClienteDemo(@Value("${cliente.service.url}") String baseUrl) {
    this.client = HttpGraphQLClient.builder()
        .webClient(b -> b.baseUrl(baseUrl))
        .build();
}
```

Consumindo APIs GraphQL

```
@Override
public void run(String... args) throws Exception {

    simplesQuery().subscribe();
}

private Mono<Void> simplesQuery(){
    String query = "query { " +
        "  todosalunos {" +
        "    idAluno " +
        "    primeiroNome " +
        "    sobrenome " +
        "    idGrupo " +
        "  }" +
        "}";

    Mono<List<AlunoDto>> mono = this.client.document(query).execute()
        .map(cr -> cr.field(path:"todosalunos").toEntityList(elementType:AlunoDto.class));
    return this.executor(message:"Um exemplo de Query simples", mono);
}

private <T> Mono<Void> executor(String message, Mono<T> mono){
    return Mono.delay(Duration.ofSeconds(seconds:1))
        .doFirst(() -> System.out.println(message))
        .then(mono)
        .doOnNext(System.out::println)
        .then();
}
```

```
1  query MyQuery {
2    todosalunos {
3      idAluno
4      primeiroNome
5      sobrenome
6      idGrupo
7    }
8  }
```



Consumindo APIs GraphQL

- `document(query)`: Informa ao cliente qual é a query GraphQL a ser enviada.
- `.execute()`: Dispara a requisição. O resultado é um `Mono<ClientResponse>` (ou similar) que contém a resposta bruta do servidor GraphQL.
- `.map(cr -> ...)`: Transforma a resposta bruta (`ClientResponse`) na lista de objetos `AlunoDto`.
 - `cr.field("todosalunos")`: Navega dentro da resposta GraphQL para o campo de dados que contém a lista (`data.todosalunos`).
 - `.toEntityList(AlunoDto.class)`: Mapeia o JSON desse campo diretamente para uma `List<AlunoDto>` (lista de objetos Java).

Consumindo APIs GraphQL

Um exemplo de Query simples

```
Este foi o documento enviado: Document{definitions=[OperationDefinition{name='null', operation=QUERY, variableDefinitions=[], directives=[], selectionSet=SelectionSet{selections=[Field{name='todosalunos', alias='null', arguments=[], directives=[], selectionSet=SelectionSet{selections=[Field{name='idAluno', alias='null', arguments=[], directives=[], selectionSet=null}, Field{name='primeiroNome', alias='null', arguments=[], directives=[], selectionSet=null}, Field{name='sobrenome', alias='null', arguments=[], directives=[], selectionSet=null}, Field{name='idGrupo', alias='null', arguments=[], directives=[], selectionSet=null}]}]}]}]}]}  
[AlunoDto[idAluno=1, primeiroNome=João, sobrenome=Silva, idGrupo=1], AlunoDto[idAluno=2, primeiroNome=Maria, sobrenome=Souza, idGrupo=2], AlunoDto[idAluno=3, primeiroNome=Thiago, sobrenome=Luiz, idGrupo=3], AlunoDto[idAluno=4, primeiroNome=Josefa, sobrenome=Barbosa, idGrupo=4], AlunoDto[idAluno=5, primeiroNome=Luiz, sobrenome=Gonzaga, idGrupo=5]]
```

```
{  
  "data": {  
    "todosalunos": [  
      {  
        "idAluno": "1",  
        "primeiroNome": "João",  
        "sobrenome": "Silva",  
        "idGrupo": 1  
      },  
      {  
        "idAluno": "2",  
        "primeiroNome": "Maria",  
        "sobrenome": "Souza",  
        "idGrupo": 2  
      },  
      {  
        "idAluno": "3",  
        "primeiroNome": "Thiago",  
        "sobrenome": "Luiz",  
        "idGrupo": 3  
      },  
      {  
        "idAluno": "4",  
        "primeiroNome": "Josefa",  
        "sobrenome": "Barbosa",  
        "idGrupo": 4  
      },  
      {  
        "idAluno": "5",  
        "primeiroNome": "Luiz",  
        "sobrenome": "Gonzaga",  
        "idGrupo": 5  
      }  
    ]  
  }  
}
```

Consumindo APIs GraphQL

- O método **.retrieve()** oferece uma abstração de alto nível destinada à maioria dos casos de uso, simplificando drasticamente o código de mapeamento.
- Em vez de retornar o `ClientResponse` completo, ele retorna um objeto `RetrieveSpec`, que permite o encadeamento imediato de métodos focados na tipagem, como `.toEntity()` (para um único objeto) ou `.toEntityList()` (para listas).
- O argumento fornecido ao `.retrieve()` (por exemplo, `.retrieve("todosalunos")`) serve como um atalho que instrui o cliente a navegar diretamente dentro do campo "data" e isolar o subcampo `todosalunos`, preparando-o para o mapeamento final.

```
public Mono<List<Aluno>> buscarTodosAlunos() {  
    String query = ""  
        query {  
            todosalunos {  
                idAluno  
                primeiroNome  
                sobrenome  
                idGrupo  
            }  
        }  
        "";  
  
    return this.client.document(query)  
        .retrieve(path:"todosalunos") // Mapeia para data.todosalunos  
        .toEntityList(elementType:Aluno.class);  
}
```

Consumindo APIs GraphQL

```
public Mono<Aluno> criarNovoAluno(AlunoDto alunoInput) {  
    String mutation = ""  
        mutation CriarNovoAluno($aluno: AlunoInput!) {  
            criarAluno(aluno: $aluno) {  
                idAluno  
                primeiroNome  
                sobrenome  
            }  
        }  
        "";  
  
    return this.client.document(mutation)  
        .variable(name:"aluno", alunoInput)  
        .retrieve(path:"criarAluno") // Mapeia para data.criarAluno  
        .toEntity(entityType:Aluno.class);  
}
```

Consumindo APIs GraphQL

```
@Service
public class ClienteDemoSubscription {
    private final WebSocketGraphQLClient client;

    public ClienteDemoSubscription(@Value("${aluno.events.subscription.url}") String baseUrl){
        this.client = WebSocketGraphQLClient.builder(baseUrl, new ReactorNettyWebSocketClient()).build();
    }

    public Flux<EventoAluno> alunoEventos(){
        var doc = "
            subscription{\n" +
            "                obtereventosAluno{\n" +
            "                    action\n" +
            "                }\n" +
            "            }";
        return this.client.document(doc)
            .retrieveSubscription(path:"obtereventosAluno")
            .toEntity(entityType:EventoAluno.class);
    }
}
```